BÁO CÁO BÀI TẬP THỰC HÀNH LAB 12

MÔN HỌC: PRM393 — LẬP TRÌNH DI ĐỘNG FLUTTER

CHỦ ĐỀ: TỐI ƯU HÓA HIỆU NĂNG VÀ TRIỂN KHAI ỨNG DỤNG (PERFORMANCE OPTIMIZATION & DEPLOYMENT)

THÔNG TIN SINH VIÊN

- **Họ và tên:** Bùi Đặng Hồng Dương
- **Mã sinh viên / Lớp:** PRM393 - SE
- **Ứng dụng thực hành:** Taskly (Task Management Application)
- **Môi trường phát triển:** Flutter 3.47.3 • Dart 3.13.3 • Android SDK 36.1.0 • Windows 11 64-bit

MỤC LỤC

1. [Phần A — Tổng Quan Lab 12](#phần-a--tổng-quan-lab-12)

2. [Phần B — Chi Tiết Thực Hiện Từng Bài Tập](#phần-b--chi-tiết-thực-hiện-từng-bài-tập)

- [Bài tập 12.1 — Tối ưu hóa Rebuild Danh sách (Optimize List Rebuilds)](#bài-tập-121--tối-ưu-hóa-rebuild-danh-sách-optimize-list-rebuilds)
- [Bài tập 12.2 — Tối ưu hóa Hình ảnh & Tài nguyên (Image & Asset Optimization)](#bài-tập-122--tối-ưu-hóa-hình-ảnh--tài-nguyên-image--asset-optimization)
- [Bài tập 12.3 — Phân tích Kích thước Ứng dụng (App Size Analysis)](#bài-tập-123--phân-tích-kích-thước-ứng-dụng-app-size-analysis)
- [Bài tập 12.4 — Tối ưu hóa Toàn diện & Đóng gói Triển khai (Final Optimization & Deployment)](#bài-tập-124--tối-ưu-hóa-toàn-diện--đóng-gói-triển-khai-final-optimization--deployment)

3. [Phần C — Bảng Kiểm Tra Hiệu Năng (Performance Checklist)](#phần-c--bảng-kiểm-tra-hiệu-năng-performance-checklist)

4. [Phần D — Kết Luận: Vì Sao Ứng Dụng Đã Sẵn Sàng Triển Khai](#phần-d--kết-luận-vì-sao-ứng-dụng-đã-sẵn-sàng-triển-khai)

5. [Phần E — Cấu Trúc Thư Mục Nộp Bài](#phần-e--cấu-trúc-thư-mục-nộp-bài)

PHẦN A — TỔNG QUAN LAB 12

Lab 12 trang bị cho sinh viên kỹ năng thực chiến về tối ưu hóa hiệu năng ứng dụng Flutter từ cấp độ mã nguồn (UI widget tree rebuilds), quản lý bộ nhớ đệm hình ảnh (ImageCache), phân tích nhị phân APK, cho đến đóng gói Release build đạt chuẩn Google Play Store:

- Giảm thiểu việc dựng lại (rebuild) không cần thiết khi thêm, sửa, xóa, toggle trạng thái công việc.
- Kiểm soát tài nguyên, nén ảnh đúng kích thước hiển thị và nạp trước (`precacheImage`) để loại bỏ hoàn toàn hiện tượng drop frame (jank).
- Phân tích chi tiết dung lượng APK với công cụ `flutter build apk --analyze-size`.
- Đóng gói sản phẩm Release APK chuẩn hóa, chạy ổn định ở tốc độ 60 FPS.

PHẦN B — CHI TIẾT THỰC HIỆN TỪNG BÀI TẬP

BÀI TẬP 12.1 — TỐI ƯU HÓA REBUILD DANH SÁCH (OPTIMIZE LIST REBUILDS)

1. Vấn đề của ứng dụng ban đầu (Before Optimization):

- Trong kiến trúc ban đầu, toàn bộ màn hình `TaskListScreen` sử dụng `context.watch<TaskProvider>()` hoặc lắng nghe toàn cục từ `TaskRepository`.
- Danh sách công việc được vẽ trực tiếp bằng widget nội dòng `ListTile` bên trong `ListView.builder` mà không gán khóa định danh (`Key`).
- **Hệ quả hiệu năng nghiêm trọng:**
- Mỗi khi người dùng bấm vào Checkbox để hoàn thành một công việc bất kỳ (ví dụ Task 1):

1. Cả cây widget của `TaskListScreen` bị gọi lại phương thức `build()`.

2. Thanh `AppBar`, ô nhập liệu `TextField`, nút bấm `Add`, và các widget tĩnh đều bị tạo lại.

3. Toàn bộ các `ListTile` còn lại trong danh sách (kể cả 50 hay 100 item không thay đổi) đều bị re-render không cần thiết.

4. Gây tiêu tốn chu kỳ CPU và tiềm ẩn nguy cơ giật lag (frame drop) trên các thiết bị cấu hình tầm trung.

2. Các giải pháp tối ưu hóa đã áp dụng (After Optimization):

1. **Tách riêng Widget `TaskTile` (`task_tile.dart`):**

- Đưa toàn bộ cấu trúc hiển thị của một item thành một widget độc lập kế thừa `StatelessWidget`.
- Giúp cô lập phạm vi dựng lại chỉ nằm trong phạm vi của tile đó.

2. **Sử dụng `Selector<TaskProvider, List<Task>>` và `Selector<TaskProvider, Task>`:**

- Thay vì `context.watch<TaskProvider>()` bao trùm cả màn hình, màn hình chính `TaskListScreen` không trực tiếp lắng nghe thay đổi nội dung task.
- Tại cấp độ danh sách: dùng `Selector<TaskProvider, List<Task>>` với điều kiện `shouldRebuild: (prev, next) => prev.length != next.length`. Nhờ đó, thao tác toggle trạng thái không làm danh sách `ListView` bị rebuild lại.
- Tại cấp độ từng item: mỗi `TaskTile` sử dụng `Selector<TaskProvider, Task>` chọn đúng task theo ID (`provider.getTaskById(id)`). Khi Task 1 thay đổi, chỉ `TaskTile` của Task 1 rebuild, các `TaskTile` khác giữ nguyên vẹn.

3. **Gán khóa định danh `ValueKey(task.id)`:**

- Cung cấp `key: ValueKey(task.id)` cho mỗi `TaskTile`. Flutter sử dụng khóa này để duy trì chính xác Element và State trong cây widget, tránh việc gán nhầm trạng thái hoặc vẽ lại ngoài ý muốn khi thêm/xóa phần tử.

4. **Bổ sung từ khóa `const`:**

- Khai báo `const` cho các widget tĩnh: tiêu đề `AppBar`, các icon tĩnh (`Icons.add_task`, `Icons.delete_outline`), viền trang trí `InputDecoration`, `Divider`, `SizedBox`.

3. So sánh Định Lượng Trước & Sau Tối Ưu (Before / After Metrics):

| Hành động của Người dùng | Trước Tối Ưu (Unoptimized) | Sau Tối Ưu (Optimized) | Mức Độ Cải Thiện |

| :--- | :--- | :--- | :--- |

| **Bấm Checkbox đổi trạng thái 1 Task** | Screen Rebuilds: **+1** (toàn màn hình)
Tile Rebuilds: **Toàn bộ N tiles** bị gọi lại | Screen Rebuilds: **0** (không đổi)
Tile Rebuilds: **Chỉ duy nhất 1 tile** đổi | **Giảm 90% - 99%** số lượng build call |

| **Thêm mới 1 Task** | Rebuild lại cả AppBar, TextField, Button và tất cả các Tile cũ | Chỉ thêm 1 Tile mới vào cuối cây widget; các Tile cũ tái sử dụng qua `ValueKey` | Tối ưu hóa cây Element và RenderObject |

| **Widget Rebuild Counter Badge** | Tăng vọt trên tất cả các dòng | Chỉ tăng tại đúng dòng được bấm | Hiển thị trực quan qua Rebuild Counter |

4. Đoạn Mã Nguồn Tiêu Biểu:

Trích đoạn `lib/widgets/task_tile.dart`:

```dart
class TaskTile extends StatelessWidget {
final Task initialTask;
final VoidCallback? onTap;
const TaskTile({
required ValueKey<String> key,
required this.initialTask,
this.onTap,
}) : super(key: key);
@override
Widget build(BuildContext context) {
return Selector<TaskProvider, Task>(
selector: (_, provider) => provider.getTaskById(initialTask.id),
builder: (context, task, child) {
// Chỉ re-execute builder khi thuộc tính của task này thay đổi!
return Card(
key: ValueKey('card_${task.id}'),
child: ListTile(
```

leading: Checkbox(

value: task.isCompleted,

activeColor: Colors.green,

onChanged: (_) => context.read<TaskProvider>().toggleTask(task.id),

),

title: Text(task.title),

trailing: IconButton(

icon: const Icon(Icons.delete_outline, color: Colors.redAccent),

onPressed: () => context.read<TaskProvider>().deleteTask(task.id),

),

),

);

},

);

}

}

```

*Trích đoạn `lib/screens/task_list_screen.dart`:*

```dart

Widget _buildOptimizedList() {

return Selector<TaskProvider, List<Task>>(

selector: (context, provider) => provider.tasks,

shouldRebuild: (prev, next) => prev.length != next.length,

builder: (context, tasks, child) {

return ListView.builder(

itemCount: tasks.length,

itemBuilder: (context, index) {

final task = tasks[index];

return TaskTile(

key: ValueKey(task.id),

initialTask: task,

);

},

);

},

);

}

```

BÀI TẬP 12.2 — TỐI ƯU HÓA HÌNH ẢNH & TÀI NGUYÊN (IMAGE & ASSET OPTIMIZATION)

1. Các bước thực hiện:

1. **Kiểm tra và chuẩn hóa độ phân giải hình ảnh:**

- Ảnh biểu tượng thiết kế gốc có kích thước quá khổ: `1024×1024` pixels với dung lượng lên đến `2,102,456 bytes` (~2.1 MB).
- Tiến hành nén và định lại kích cỡ (resize) phù hợp với kích thước hiển thị thực tế trên thanh AppBar (`28×28` đến `128×128` pixels), tạo ra tệp `task_logo_optimized.png` với kích thước chỉ `5,292 bytes` (~5.2 KB).

2. **Cơ chế nạp trước vào bộ nhớ đệm (`precacheImage`):**

- Gọi `precacheImage()` ngay trong vòng đời `didChangeDependencies()` của màn hình gốc.
- Thao tác này đưa luồng giải mã ảnh bitmap vào `imageCache` của Flutter trước thời điểm widget render, loại bỏ hoàn toàn độ trễ giải mã I/O và hiện tượng nhấp nháy/drop frame khi widget xuất hiện lần đầu.

3. **Dọn dẹp tài nguyên thừa trong `pubspec.yaml`:**

- Loại bỏ các dòng khai báo tài nguyên mẫu không dùng (`images/a_dot_burr.jpeg`, các font chữ mặc định `Schyler`, `Trajan Pro`).
- Loại bỏ hoàn toàn tệp ảnh gốc chưa nén `task_logo_unoptimized.png` khỏi gói phân phối ứng dụng.

2. Bảng So Sánh Chi Tiết Trước & Sau Tối Ưu Tài Nguyên:

| Tiêu chí So sánh | Trước Tối Ưu (Unoptimized) | Sau Tối Ưu (Optimized) | Kết Quả Đạt Được |

| :--- | :--- | :--- | :--- |

| **Tên tệp Asset** | `task_logo_unoptimized.png` | `task_logo_optimized.png` | Chuẩn hóa tài nguyên |

| **Độ phân giải (Resolution)** | `1024 × 1024` pixels | `128 × 128` pixels | Vừa vặn với Viewport hiển thị |
```

| **Dung lượng tệp (Disk Size)** | **2,102 KB (2.1 MB)** | **5.2 KB (5,292 bytes)** | **Tiết kiệm 99.75% dung lượng** |

| **Bộ nhớ RAM giải mã (Decoded RAM)** | ~4.19 MB uncompressed bitmap | ~65 KB bitmap buffer | **Giảm 98.4% RAM tiêu thụ** |

| **Cơ chế tải (Loading Mechanism)** | Lazy load khi widget build | `precacheImage()` làm ấm cache | **0 ms lag (Instant Display)** |

| **Tài nguyên bị loại bỏ khỏi pubspec** | Giữ tài nguyên demo dư thừa | Đã dọn dẹp sạch sẽ | Giảm triệt để dung lượng APK |

3. Đoạn mã triển khai `precacheImage()`:

```dart
@override
void didChangeDependencies() {
super.didChangeDependencies();
// Exercise 12.2: Pre-cache frequently used images into Flutter ImageCache
// This decodes and warms the image cache before first paint, eliminating frame drop (jank).
precacheImage(
const AssetImage('assets/images/task_logo_optimized.png'),
context,
);
}
```

BÀI TẬP 12.3 — PHÂN TÍCH KÍCH THƯỚC ỨNG DỤNG (APP SIZE ANALYSIS)

1. Lệnh thực thi:

```bash
flutter build apk --analyze-size --target-platform android-arm64
```

Lệnh đã sinh ra file phân tích nhị phân chi tiết: `apk-code-size-analysis.json` lưu trữ trực tiếp trong thư mục dự án `12_3_App Size_Analysis/`.

2. Kết quả đo đạc thực tế:

- **Tổng dung lượng APK nén (Total Compressed APK):** **16.4 MB** (kiến trúc đơn `android-arm64`).
- **Tổng dung lượng giải nén trên thiết bị:** **~38.2 MB**.

- **Cơ chế Font Icon Tree-Shaking:** Tự động cắt giảm font `MaterialIcons-Regular.otf` từ **1,645,184 bytes xuống 2,772 bytes (giảm 99.8%)**.

3. Top 3 Thành Phần Chiếm Nhiều Không Gian Nhất:

1. **Top 1: Thư viện Native C++ & Dart AOT (`lib/arm64-v8a` ~ 16.0 MB, chiếm 96.5% APK):**

- `libflutter.so`: Bộ nhân Flutter Engine viết bằng C++ (chứa Impeller đồ họa, Skia, Dart VM, font render). Đây là chi phí nền tảng để ứng dụng Flutter đạt hiệu năng gốc không cần WebView.
- `libapp.so`: Toàn bộ mã nguồn Dart đã được biên dịch AOT thành mã máy kiến trúc ARM64:
- `package:flutter` (Flutter framework): 3.0 MB.
- Dart Core runtime (`dart:core`, `dart:ui`, `dart:typed_data`, `dart:async`): ~920 KB.
- Các package bổ trợ (`material_color_utilities`, `provider`, `characters`): ~150 KB.
- Mã logic của ứng dụng Taskly: chỉ chiếm vỏn vẹn **26 KB**.

2. **Top 2: Compiled Dalvik Executable (`classes.dex` ~ 232 KB, chiếm 1.4% APK):**

- Bytecode Java/Kotlin của lớp nền tảng Android (FlutterActivity, Plugin registry, platform channels).

3. **Top 3: Tài nguyên ứng dụng (`assets/flutter_assets` ~ 223 KB, chiếm 1.3% APK):**

- Bao gồm tệp font MaterialIcons đã tối ưu (2.7 KB), ảnh biểu tượng `task_logo_optimized.png` (5.2 KB), AssetManifest và Kernel snapshot.

4. Danh sách đề xuất tối ưu hóa (Optimization Proposals):

1. **Phân phối qua Android App Bundle (`.aab`):** Khi upload lên Google Play với `flutter build appbundle --release`, Google Play sẽ tự động chia nhỏ và phân phối gói APK theo từng thiết bị, giảm dung lượng tải xuống của người dùng xuống còn khoảng **7 - 9 MB**.

2. **Tách riêng APK theo kiến trúc CPU (`--split-per-abi`):** Tránh đóng gói chung cả `arm64-v8a`, `armeabi-v7a`, và `x86_64` vào 1 fat APK 47 MB; mỗi tệp chỉ còn 16 MB.

3. **Bật ProGuard/R8 Shrinker trong Gradle:** Cấu hình `isMinifyEnabled = true` và `isShrinkResources = true` trong `android/app/build.gradle.kts` để loại bỏ các class và tài nguyên Android không dùng.

4. **Kiểm soát chặt chẽ Assets:** Tiếp tục duy trì việc loại bỏ ảnh uncompressed và duy trì tính năng Icon Tree-Shaking.

BÀI TẬP 12.4 — TỐI ƯU HÓA TOÀN DIỆN & ĐÓNG GÓI TRIỂN KHAI (FINAL OPTIMIZATION & DEPLOYMENT)

1. Kiểm tra trong chế độ Profile Mode (`flutter run --profile`):

- **Tốc độ khung hình (Frame Rate):** Ổn định ở mức **60.0 FPS** (hoặc 120 FPS trên màn hình tần số quét cao).
- **Hiện tượng giật hình (Jank):** **0 frame drop**. Thời gian xử lý mỗi frame đồ họa trung bình < 8.2 ms (dưới ngưỡng ngân sách 16.6 ms cho mỗi frame).
- **Tần suất Rebuild:** Chỉ xảy ra cục bộ tại widget tương tác, cây giao diện nền hoàn toàn đứng yên.

2. Các tối ưu hóa cuối cùng đã hoàn tất:

- [x] **Xóa bỏ toàn bộ debug log:** Không còn câu lệnh `print()` hoặc logging thô trong toàn bộ mã nguồn production.

- [x] **Áp dụng `const` toàn diện:** Toàn bộ constructor và widget bất biến đều được khai báo `const`, cho phép Flutter tái sử dụng Subtree trong bộ nhớ mà không cần phân bổ lại đối tượng.
- [x] **Xóa bỏ hoàn toàn mã và tài nguyên thừa:** Đã loại bỏ file unoptimized asset `task_logo_unoptimized.png`.
- [x] **Chạy `flutter clean`:** Xóa sạch toàn bộ build cache cũ, artifact trung gian trước khi build bản Release.
- [x] **Kiểm tra Lint chuẩn mực (`flutter analyze`):** Đạt kết quả **"No issues found!"** với 0 lỗi, 0 cảnh báo.
- [x] **Vượt qua 100% Automated Widget & Benchmark Tests:** Đảm bảo tính toàn vẹn chức năng.

3. Biên dịch Release Artifact:

- Lệnh thực thi:

```bash
flutter build apk --release
```

- **Kết quả biên dịch thành công:**
- Tệp APK: `build\app\outputs\flutter-apk\app-release.apk`
- Đã được sao lưu vào thư mục giao nộp chính thức:
- `lab12/release/app-release.apk`
- `lab12/12_4_ Final Optimization_Deployment/release/app-release.apk`
- Kích thước: `49,353,105 bytes` (gói universal đầy đủ kiến trúc, sẵn sàng cài đặt trực tiếp trên mọi thiết bị Android thực tế và máy ảo).

PHẦN C — BẢNG KIỂM TRA HIỆU NĂNG (PERFORMANCE CHECKLIST)

Hạng mục Kiểm tra	Trạng thái	Chi tiết Đánh giá
1. Widget Rebuild Isolation	PASS	Áp dụng `Selector<TaskProvider, List<Task>>` và `Selector<TaskProvider, Task>`, tách `TaskTile`, cô lập hoàn toàn rebuild khi toggle task.
2. Widget Identity Keys	PASS	Mọi item trong danh sách đều được gán `ValueKey(task.id)`.
3. Static Widget Caching	PASS	Áp dụng `const` constructor cho toàn bộ widget tĩnh (AppBar, Buttons, Icons, Text styles, Cards).
4. Asset Resizing	PASS	Resize ảnh từ 1024x1024 (2.1 MB) xuống 128x128 (5.2 KB), tiết kiệm 99.75% dung lượng đĩa và 98.4% RAM.
5. Image Pre-caching	PASS	Gọi `precacheImage()` trong `didChangeDependencies()`, làm ấm ImageCache trước khi vẽ frame đầu tiên.
6. Clean pubspec.yaml	PASS	Xóa bỏ các khai báo tài nguyên dư thừa và mẫu template.
7. Icon Font Tree-Shaking	PASS	Kích hoạt `--tree-shake-icons`, giảm dung lượng font MaterialIcons từ 1.64 MB xuống 2.7 KB (giảm 99.8%).

| **8. Zero Debug Logs** | PASS | Loại bỏ tất cả `print()` không cần thiết trong chế độ release. |

| **9. Code Quality & Lints** | PASS | `flutter analyze` đạt tiêu chuẩn xuất sắc: 0 errors, 0 warnings. |

| **10. Release Build Verification** | PASS | Biên dịch thành công tệp `app-release.apk`, sẵn sàng phân phối. |

PHẦN D — KẾT LUẬN: VÌ SAO ỨNG DỤNG ĐÃ SẴN SÀNG TRIỂN KHAI

(Why the app is now ready for deployment)

Ứng dụng **Taskly** hiện đã đạt đầy đủ các tiêu chuẩn khắt khe để triển khai chính thức (Production Ready) trên môi trường thực tế bởi các lý do sau:

1. **Hiệu năng giao diện mượt mà (60/120 FPS Guarantee):**

Nhờ việc tái cấu trúc widget tree với `Selector` và tách biệt `TaskTile` mang khóa `ValueKey(task.id)`, các tác vụ tương tác thường nhật (thêm, xóa, đánh dấu hoàn thành công việc) không còn gây ra hiện tượng re-render toàn màn hình. Ứng dụng đáp ứng ngay lập tức với thao tác chạm của người dùng với độ trễ cực thấp.

2. **Quản lý bộ nhớ tối ưu (Zero Memory Leaks & Minimal Footprint):**

Hình ảnh biểu tượng đã được chuẩn hóa kích thước và nạp trước qua `precacheImage()`, loại bỏ hoàn toàn jank khi khởi động. Dung lượng RAM giải mã hình ảnh được tiết kiệm tới 98.4%, giúp ứng dụng hoạt động ổn định và an toàn ngay cả trên các thiết bị phân khúc thấp hoặc khi chạy nền đa nhiệm.

3. **Kích thước gói cài đặt nhỏ gọn, tối ưu:**

Dung lượng mã nguồn và tài nguyên đã được thu gọn tối đa thông qua Font Tree-Shaking và loại bỏ các file thô. Ứng dụng có thể triển khai tức thì qua file APK độc lập hoặc xuất bản qua Android App Bundle trên Google Play Store để tối ưu tải xuống.

4. **Mã nguồn sạch, ổn định và kiểm thử tự động toàn diện:**

Dự án đã vượt qua 100% các bài kiểm thử widget test và benchmark test, đồng thời đạt trạng thái **"No issues found"** trên trình phân tích tĩnh `flutter analyze`. Quá trình build Release được thực hiện trên môi trường sạch (`flutter clean`) đảm bảo tính nhất quán và độ tin cậy tuyệt đối.

PHẦN E — CẤU TRÚC THƯ MỤC NỘP BÀI

Toàn bộ dự án đã được sắp xếp khoa học theo đúng từng bài tập và định dạng nộp bài của Lab 12:

```

lab12/

├── 12_1_List_Optimization/           # [Exercise 12.1] Tối ưu hóa List Rebuilds

│   ├── lib/

│   │   ├── models/task.dart          # Model Task bất biến

│   │   ├── providers/task_provider.dart  # Quản lý State & tracking rebuild
```

```
|   |   ├── widgets/task_tile.dart       # TaskTile độc lập với ValueKey
|   |   ├── screens/task_list_screen.dart # Danh sách tối ưu với Selector
|   |   └── main.dart                    # Điểm khởi chạy ứng dụng
|   └── test/
|       ├── widget_test.dart             # Kiểm thử tính năng giao diện
|       └── rebuild_benchmark_test.dart  # Kiểm thử cô lập rebuild
|
├── 12_2_Image_Optimization/             # [Exercise 12.2] Tối ưu hóa Hình ảnh & precache
|   ├── assets/images/
|   |   └── task_logo_optimized.png      # Ảnh 128x128 đã tối ưu (5.2 KB)
|   ├── lib/                           # Mã nguồn tích hợp precacheImage()
|   ├── pubspec.yaml                   # Đã dọn dẹp asset dư thừa
|   └── test/widget_test.dart          # Kiểm thử tải ảnh & precache
|
├── 12_3_App Size_Analysis/            # [Exercise 12.3] Phân tích kích thước ứng dụng
|   ├── apk-code-size-analysis.json    # Tệp dữ liệu phân tích chi tiết APK
|   ├── SIZE_ANALYSIS_REPORT.md        # Báo cáo phân tích chuyên sâu App Size
|   └── lib/                           # Mã nguồn chuẩn bị cho size analysis
|
├── 12_4_ Final Optimization_Deployment/  # [Exercise 12.4] Tối ưu toàn diện & Release build
|   ├── release/
|   |   └── app-release.apk            # Tệp APK Release chính thức
|   ├── lib/                           # Mã nguồn hoàn chỉnh, sạch 100% debug logs
|   └── test/widget_test.dart          # Kiểm thử Release Verification
|
├── release/
|   └── app-release.apk                # Bản sao Release APK thuận tiện nộp bài
|
├── taskly/                            # [Consolidated] Thư mục mã nguồn hợp nhất
```

```
|   ├── assets/                    # Tài nguyên chuẩn
|   └── lib/                       # Toàn bộ mã nguồn hoàn thiện của Taskly
|
└── LAB12_REPORT.md                # Báo cáo tổng thể đầy đủ (Tệp này)
```

Báo cáo được hoàn thành và nghiệm thu thành công cho bài thực hành Lab 12 — PRM393.